

TALLER

ORACLE

BASE DE DATOS

Taller Práctico Oracle PL/SQL

Desarrollar aplicaciones con Oracle PL/SQL

Taller práctico para fortalecer tus habilidades en programación de bases de datos, optimizar procesos y crear soluciones confiables y escalables con las mejores prácticas de Oracle PL/SQL.

Relator: Claudio Gutierrez (c.gutierrezhartard@tcs.com)



Agenda

01

Introducción a PL/SQL

Fundamentos, contexto y objetivos del taller.

03

Índices y Consultas SQL

Optimización básica y consultas para recuperar información.

05

Oracle SQL Developer

Uso de la herramienta para desarrollar, probar y depurar.

07

Mejores Prácticas

Recomendaciones para escribir código claro, seguro y mantenible.

02

Modelo de Datos y Tablas

Estructura relacional, relaciones y diseño de tablas.

04

Procedimientos, Funciones y Reportes

Creación de lógica reutilizable y generación de salidas útiles.

06

Ejercicios Prácticos

Aplicación guiada de los conceptos en casos reales.

08

Conclusión

Resumen final y espacio para dudas.

¿Qué es PL/SQL?

Procedural Language/SQL es una potente extensión de lenguaje, desarrollada por Oracle. Combina la capacidad de manipulación de datos de SQL con las características de programación de un lenguaje procedural, permitiendo desarrollar aplicaciones robustas y optimizadas directamente dentro de la base de datos Oracle.

Programación Procedimental

Añade estructuras de control como bucles, condiciones y funciones para una lógica de negocio compleja.

Integración Transparente

Ejecuta sentencias SQL de forma nativa y optimizada, mejorando la interacción con los datos.

Rendimiento Optimizado

Reduce el tráfico de red y el procesamiento en el servidor, agilizando las operaciones y consultas.

Sintaxis Básica de PL/SQL

La estructura básica de un bloque PL/SQL se compone de varias secciones, cada una con un propósito específico para organizar y ejecutar el código.

```
DECLARE -- Declaración de variables
    v_nombre VARCHAR2(50); v_edad NUMBER; v_fecha DATE := SYSDATE;
BEGIN -- Lógica principal
    v_nombre := 'Juan';
    v_edad := 30;
    IF v_edad >= 18 THEN
        DBMS_OUTPUT.PUT_LINE(v_nombre || ' es mayor de edad');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_nombre || ' es menor de edad');
    END IF;
EXCEPTION -- Manejo de errores
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Sintaxis Básica de PL/SQL

A continuación, se explica cada sección fundamental de un bloque PL/SQL:

01

DECLARE (Opcional)

Esta sección se utiliza para declarar variables, constantes, cursores, tipos definidos por el usuario y excepciones. Aquí se especifican los **tipos de datos** (como VARCHAR2, NUMBER, DATE) que se usarán en el bloque.

03

EXCEPTION (Opcional)

Sección dedicada al manejo de errores. Permite interceptar y gestionar las excepciones (errores) que pueden ocurrir durante la ejecución de la sección BEGIN. La cláusula WHEN OTHERS THEN captura cualquier error no manejado específicamente, y SQLERRM proporciona el mensaje del error.

02

BEGIN (Obligatorio)

Contiene las sentencias ejecutables y la lógica principal del programa. Aquí se incluyen las sentencias SQL (INSERT, UPDATE, DELETE, SELECT INTO), llamadas a procedimientos y funciones, y **estructuras de control** como IF-THEN-ELSE y bucles (LOOP, FOR, WHILE).

04

END (Obligatorio)

Marca el final del bloque PL/SQL. Cada bloque DECLARE/BEGIN debe terminar con un END.

Variables y Tipos de Datos

En PL/SQL, las variables se utilizan para almacenar temporalmente valores que se pueden manipular durante la ejecución de un programa. Estos son algunos de los tipos de datos más comunes en PL/SQL:

Tipo de Dato	Descripción
VARCHAR2	Cadenas de texto de longitud variable, ideales para nombres, descripciones cortas, etc.
NUMBER	Números enteros y decimales. Adecuado para cálculos financieros o cantidades.
DATE	Almacena fechas y horas (año, mes, día, hora, minuto, segundo).
BOOLEAN	Valores verdadero (TRUE), falso (FALSE) o nulo (NULL). Utilizado para lógica condicional.
CLOB	Carácter grande (Character Large Object). Almacena grandes volúmenes de texto, hasta 4 GB.
BLOB	Binario grande (Binary Large Object). Almacena datos binarios como imágenes, audio o video, hasta 4 GB.

Variables y Tipos de Datos

La declaración de variables se realiza en la sección `DECLARE`. Es una buena práctica usar el prefijo `v_` para las variables.

```
DECLARE -- Declaración e inicialización de variables
v_nombre_empleado VARCHAR2(100) := 'Maria Lopez';
v_salario          NUMBER(10, 2) := 5500.75;
v_fecha_contrato   DATE          := SYSDATE;
v_activo           BOOLEAN       := TRUE;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Nombre: ' || v_nombre_empleado);
  DBMS_OUTPUT.PUT_LINE('Salario: ' || v_salario);
  DBMS_OUTPUT.PUT_LINE('Fecha Contrato: ' || TO_CHAR(v_fecha_contrato, 'DD-MON-YYYY'));
  IF v_activo THEN
    DBMS_OUTPUT.PUT_LINE(v_nombre_empleado || ' está activo.');
```

```
ELSE
  DBMS_OUTPUT.PUT_LINE(v_nombre_empleado || ' no está activo.');
```

```
END IF;
END;
```

Estructuras de Control en PL/SQL

A continuación se presentan algunas de las estructuras de control mas utilizadas en la programación en PL/SQL.

```
-- CONDICIONAL IF-THEN-ELSE
DECLARE
    v_edad NUMBER := 25;
BEGIN
    IF v_edad < 18 THEN
        DBMS_OUTPUT.PUT_LINE('Menor de edad');
    ELSIF v_edad < 65 THEN
        DBMS_OUTPUT.PUT_LINE('Edad laboral');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Jubilado');
    END IF;
END;
```


Estructuras de Control en PL/SQL

-- CICLOS LOOP (Bucle finito)

DECLARE

v_contador NUMBER := 1;

BEGIN

LOOP

DBMS_OUTPUT.PUT_LINE('Iteración: ' || v_contador);

v_contador := v_contador + 1;

EXIT WHEN v_contador > 5;

END LOOP;

END;

-- FOR LOOP

BEGIN

FOR i IN 1..10 LOOP

DBMS_OUTPUT.PUT_LINE('Número: ' || i);

END LOOP;

END;

Estructuras de Control en PL/SQL

```
-- WHILE LOOP
DECLARE
    v_contador NUMBER := 1;
BEGIN
    WHILE v_contador <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE('Contador: ' || v_contador);
        v_contador := v_contador + 1;
    END LOOP;
END;
/
```

Estructuras de Control en PL/SQL

Las estructuras de control son fundamentales en PL/SQL para dictar el flujo de ejecución de un programa, permitiendo la toma de decisiones y la repetición de tareas.

01

IF-THEN-ELSIF-ELSE

Utilizada para ejecutar bloques de código condicionalmente. Evalúa una o más condiciones y ejecuta el código asociado a la primera condición que sea verdadera. Es ideal para la lógica de negocio basada en decisiones.

03

FOR LOOP

Perfecto para iterar un número predefinido de veces. Se utiliza comúnmente para recorrer rangos de números o para procesar conjuntos de resultados de consultas (con cursores implícitos).

02

LOOP (Bucle Básico)

Crea un bucle infinito que debe ser terminado explícitamente con una sentencia `EXIT` o `EXIT WHEN`. Es útil cuando el número de iteraciones no se conoce de antemano o la condición de salida es compleja.

04

WHILE LOOP

Ejecuta un bloque de código repetidamente mientras una condición específica sea verdadera. La condición se evalúa al inicio de cada iteración, lo que significa que el bucle no se ejecutará si la condición inicial es falsa.

Cursores en PL/SQL

Los cursores son mecanismos que PL/SQL utiliza para procesar conjuntos de filas devueltos por una consulta SQL. Permiten trabajar con los resultados de una sentencia SELECT fila por fila, lo cual es fundamental para la lógica de negocio que necesita manipular datos de manera individualizada.

-- Cursor explícito

```
DECLARE
  CURSOR c_libros IS
    SELECT id_libro, titulo, año_publicacion
    FROM LIBRO
    WHERE año_publicacion > 1970;
  v_id_libro LIBROS.id_libro%TYPE;
  v_titulo   LIBROS.titulo%TYPE;
  v_año      LIBROS.año_publicacion%TYPE;
BEGIN
  OPEN c_libros;
  LOOP
    FETCH c_libros INTO v_id_libro, v_titulo, v_año;
    EXIT WHEN c_libros%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('ID: ' || v_id_libro || '-' || v_titulo || '-' || v_año);
  END LOOP;
  CLOSE c_libros;
END;
```

-- Cursor implícito con FOR LOOP

```
DECLARE
  CURSOR c_usuarios IS
    SELECT id_usuario, nombre, email FROM USUARIOS;
BEGIN
  FOR v_usuario IN c_usuarios LOOP
    DBMS_OUTPUT.PUT_LINE('Usuario: ' || v_usuario.nombre || '(' || v_usuario.email || ')');
  END LOOP;
END;
```

Cursores en PL/SQL

Existen dos tipos principales de cursores en PL/SQL: explícitos e implícitos. Comprender sus diferencias y cuándo usar cada uno es clave para escribir código eficiente y robusto.



Cursores Explícitos

Los cursores explícitos son definidos y controlados manualmente por el programador. Permiten un control granular sobre cada etapa del procesamiento de filas: declaración, apertura, obtención y cierre. Son ideales para consultas que devuelven múltiples filas, especialmente cuando se necesita procesar cada fila individualmente o cuando se desea mayor control y eficiencia para grandes conjuntos de datos.



Cursores Implícitos

Los cursores implícitos son creados y gestionados automáticamente por PL/SQL para sentencias DML (INSERT, UPDATE, DELETE) y para sentencias `SELECT INTO` que se espera que devuelvan una única fila. No requieren declaración, apertura, obtención o cierre manual. Son convenientes para operaciones que afectan una sola fila o para `SELECT INTO` cuando se espera un único resultado. Los bucles `FOR` de cursor también utilizan cursores implícitos, simplificando el manejo de múltiples filas.

Caso de Uso: Sistema de Gestión de Biblioteca

Este caso de uso presenta el desarrollo de un **Sistema de Gestión de Biblioteca**, una solución pensada para administrar de forma ordenada y eficiente libros, autores, usuarios y préstamos.

Catálogo y búsqueda

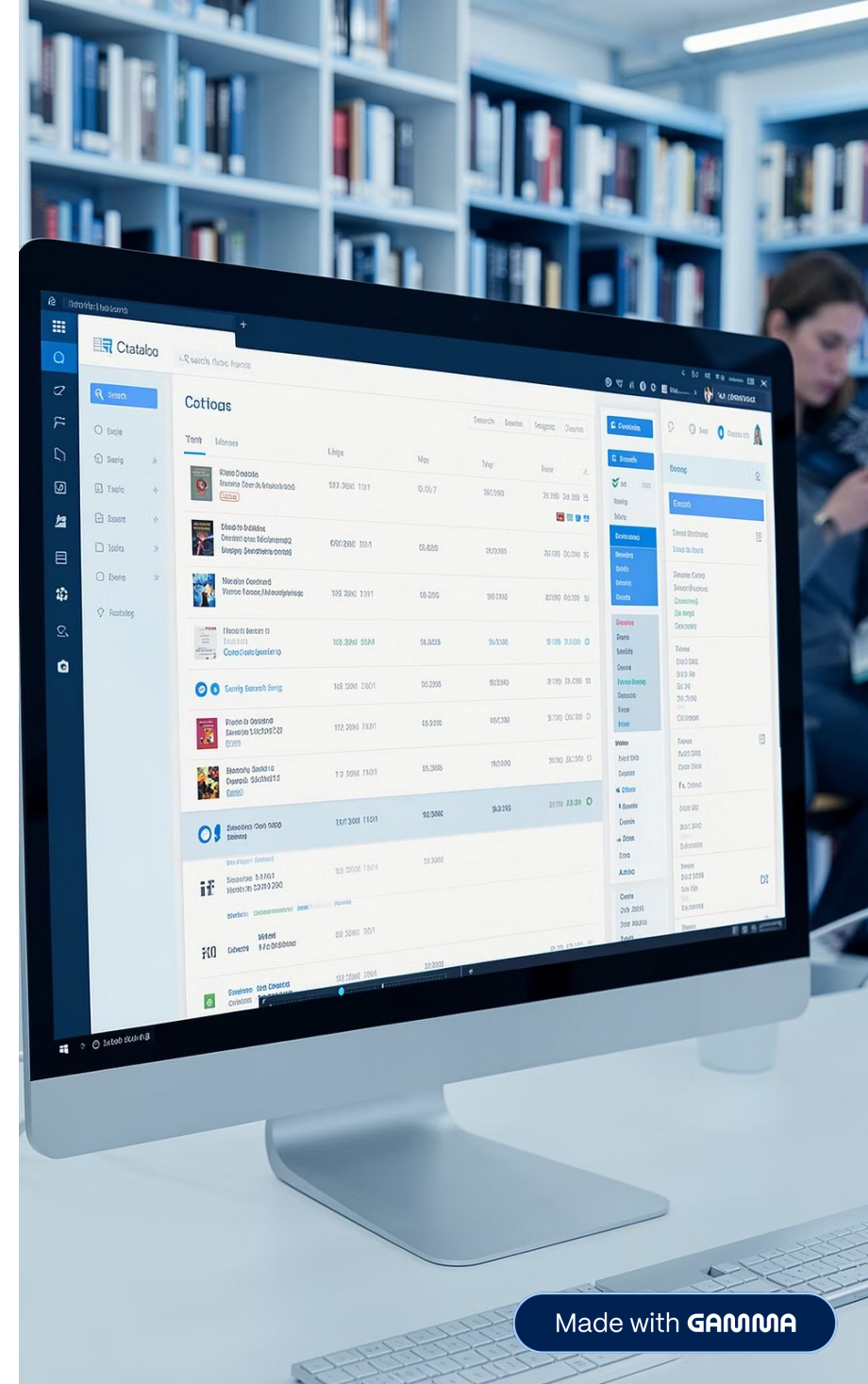
Organizar y localizar libros de manera rápida y sencilla.

Préstamos y devoluciones

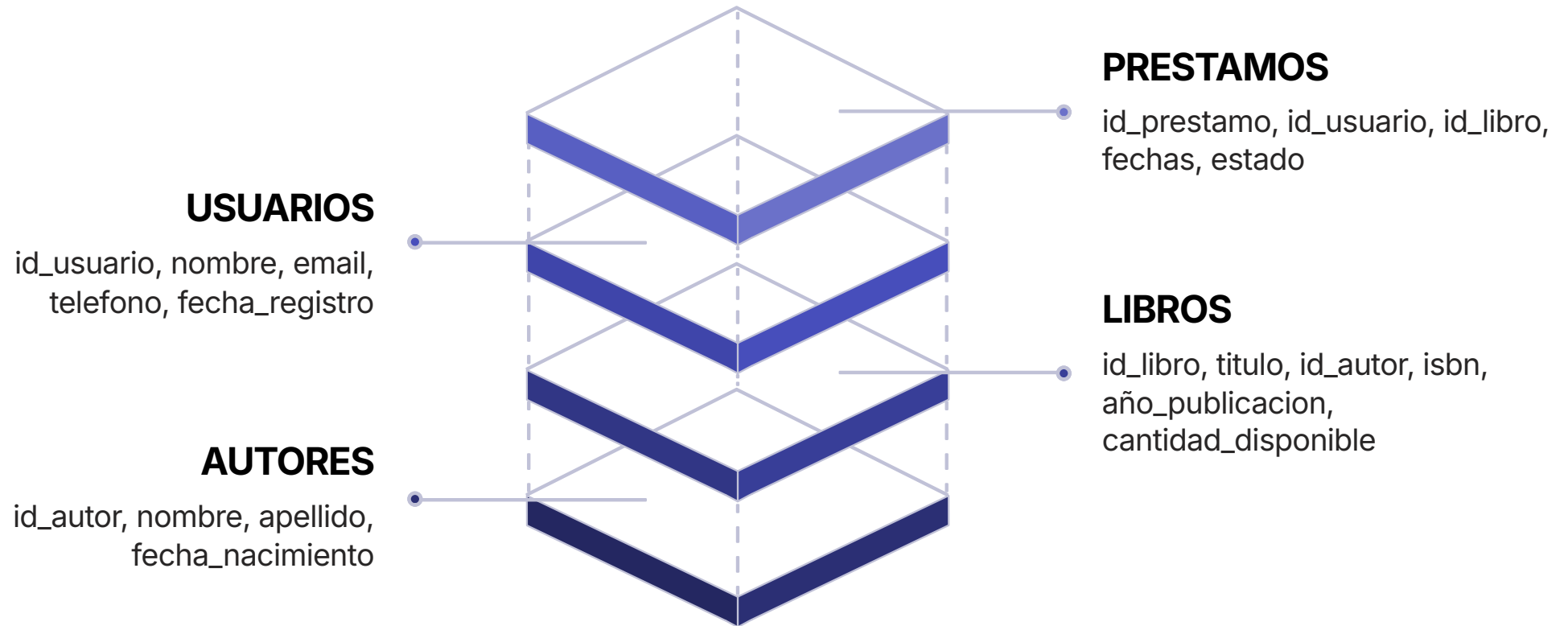
Controlar el flujo de circulación de ejemplares con mayor eficiencia.

Gestión de usuarios y autores

Mantener información actualizada y centralizada para una mejor administración.

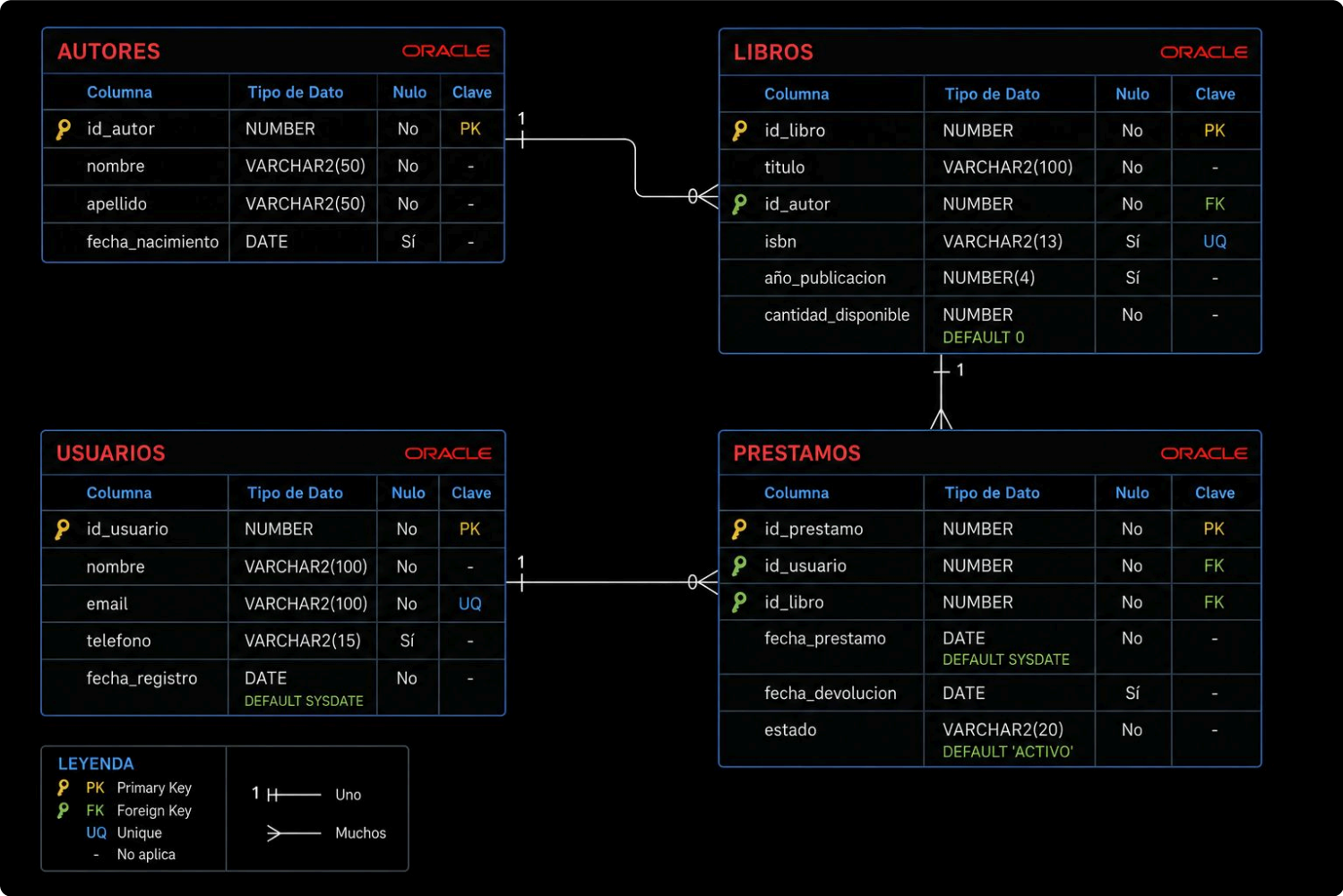


Modelo de Datos - Diagrama Entidad-Relación



Este diagrama visualiza la estructura fundamental de la base de datos de nuestro Sistema de Gestión de Biblioteca, mostrando las entidades clave y cómo se interrelacionan para mantener la integridad de la información.

Modelo de Datos Relacional



En la imagen se presenta las entidades involucradas en el modelo.

Las líneas conectoras representan la relación existente entre las distintas entidades.

Las relaciones se interpretan de la siguiente manera:

Un autor puede escribir muchos libros.

Un libro puede ser prestado muchas veces.

Un usuario puede solicitar muchos prestamos.

Indices

En el mundo de las base de datos constantemente va a escuchar hablar de consultas, indices y rendimiento.

Un indice es básicamente un acceso directo a un registro específico en la base de datos de la misma forma que lo hace un indice de un libro cualquiera.

TABLA CLIENTE

ROWID	RUT_CLIENTE	NOMBRE	RUBRO	ID_ZONA	FECHA_CREACION	ID_USUARIO
AASpxiAACAAF6DvAAA	101010	Sra Martita	(null)	1	7/24/2026, 5:14:31	OP01
AASpxiAACAAF6DvAAB	101011	Don Ramon	(null)	1	7/24/2026, 5:14:31	OP01
AASpxiAACAAF6DvAAC	101012	Doña Julia	(null)	1	7/24/2026, 5:14:31	OP01
AASpxiAACAAF6DvAAD	801001	Huerto Puro	Agricultura	2	7/24/2026, 5:14:31	OP01

Dirección física Campo indexado

INDICE CLIENTE RUT

ROWID	RUT_CLIENTE
AASpxiAACAAF6DvAAA	101010
AASpxiAACAAF6DvAAB	101011
AASpxiAACAAF6DvAAC	101012
AASpxiAACAAF6DvAAD	801001

Suponga que en su modelo tiene una tabla que almacena sus clientes y el campo RUT es un identificador principal, muchas veces se requiere buscar a los clientes por RUT.

En este caso el campo RUT debe ser indexado para mejorar el rendimiento de las búsquedas.

Así cuando se ejecute una consulta de cliente por su RUT el motor de base de datos va a recurrir al indice existente por dicha columna para encontrar el registro de forma más rápida y eficiente,.

Creación de Tablas - Parte 1

Código SQL para crear las tablas AUTORES y LIBROS, fundamentales para nuestro sistema de gestión de biblioteca.

Ingresa a la plataforma de Oracle para proceder a crear las tablas del modelo, de acuerdo al manual proporcionado <https://www.oracle.com/database/technologies/oracle-free-sql/> **FreeSQL**

```
-- Tabla AUTORES
CREATE TABLE AUTOR (
  id_autor NUMBER PRIMARY KEY,
  nombre VARCHAR2(50) NOT NULL,
  apellido VARCHAR2(50) NOT NULL,
  fecha_nacimiento DATE
);

-- Tabla LIBROS
CREATE TABLE LIBRO (
  id_libro NUMBER PRIMARY KEY,
  titulo VARCHAR2(100) NOT NULL,
  id_autor NUMBER NOT NULL,
  isbn VARCHAR2(13) UNIQUE,
  año_publicacion NUMBER(4),
  cantidad_disponible NUMBER DEFAULT 0,
  CONSTRAINT fk_libro_autor FOREIGN KEY (id_autor) REFERENCES AUTORES(id_autor)
);
```

En este código, utilizamos diversos tipos de datos: **NUMBER** para identificadores numéricos y años, **VARCHAR2** para cadenas de texto como nombres y títulos, y **DATE** para fechas. Las tablas incluyen restricciones clave como **PRIMARY KEY** para asegurar la unicidad de los registros, **NOT NULL** para campos obligatorios, y **UNIQUE** para garantizar que el ISBN sea único. Además, la tabla **LIBROS** establece una **FOREIGN KEY** (**fk_libros_autores**) que vincula cada libro con un autor existente, manteniendo la integridad referencial de los datos. Se define un valor **DEFAULT** para la cantidad de libros disponibles, estableciéndola en 0 por defecto.

Creación de Tablas - Parte 2

A continuación, se muestra el código SQL para crear las tablas `USUARIOS` y `PRESTAMOS`, que completan la estructura de nuestro sistema de gestión de biblioteca.

-- Tabla USUARIOS

```
CREATE TABLE USUARIO (  
  id_usuario NUMBER PRIMARY KEY,  
  nombre VARCHAR2(100) NOT NULL,  
  email VARCHAR2(100) UNIQUE NOT NULL,  
  telefono VARCHAR2(15),  
  fecha_registro DATE DEFAULT SYSDATE  
);
```

-- Tabla PRESTAMOS

```
CREATE TABLE PRESTAMO (  
  id_prestamo NUMBER PRIMARY KEY,  
  id_usuario NUMBER NOT NULL,  
  id_libro NUMBER NOT NULL,  
  fecha_prestamo DATE DEFAULT SYSDATE,  
  fecha_devolucion DATE,  
  estado VARCHAR2(20) DEFAULT 'ACTIVO',  
  CONSTRAINT fk_prestamo_usuario FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario),  
  CONSTRAINT fk_prestamos_libro FOREIGN KEY (id_libro) REFERENCES LIBRO(id_libro)  
);
```

Las tablas `USUARIO` y `PRESTAMO` son esenciales para gestionar los miembros de la biblioteca y sus transacciones de libros. La tabla `USUARIO` incluye `id_usuario` como `PRIMARY KEY`, campos `NOT NULL` para `nombre` y `email`, y una restricción `UNIQUE` para el `email`, asegurando que cada usuario tenga una dirección de correo electrónico única. La columna `fecha_registro` utiliza `SYSDATE` para registrar automáticamente la fecha de creación del usuario.

La tabla `PRESTAMOS` registra cada préstamo de libro, con `id_prestamo` como `PRIMARY KEY`. Incorpora dos `FOREIGN KEYs`: `fk_prestamos_usuarios` vincula cada préstamo a un `id_usuario` existente en la tabla `USUARIOS`, y `fk_prestamos_libros` lo relaciona con un `id_libro` de la tabla `LIBROS`. Las columnas `fecha_prestamo` y `estado` tienen valores `DEFAULT` para agilizar el registro y mantener la coherencia. Estas interconexiones garantizan la integridad referencial y facilitan una gestión precisa de las operaciones de la biblioteca.

Creación de Índices

Muestra el código SQL para crear índices que optimicen las consultas:

```
-- Índices para mejorar el rendimiento de búsquedas
CREATE INDEX idx_libro_autor ON LIBRO(id_autor);
CREATE INDEX idx_libro_isbn ON LIBRO(isbn);
CREATE INDEX idx_usuario_email ON USUARIO(email);
CREATE INDEX idx_prestamo_usuario ON PRESTAMO(id_usuario);
CREATE INDEX idx_prestamo_libro ON PRESTAMO(id_libro);
CREATE INDEX idx_prestamo_estado ON PRESTAMO(estado);
CREATE INDEX idx_prestamo_fecha ON PRESTAMO(fecha_prestamo);
```

Estos índices se crean para mejorar significativamente el rendimiento de las consultas en nuestro sistema de gestión de biblioteca. Se indexan columnas clave como `id_autor` e `isbn` en la tabla `LIBROS`, `email` en `USUARIOS`, y `id_usuario`, `id_libro`, `estado` y `fecha_prestamo` en la tabla `PRESTAMOS`.

La creación de índices permite que la base de datos encuentre filas de datos más rápidamente, de manera similar a cómo un índice en un libro te ayuda a encontrar información específica sin tener que leer cada página. Esto es especialmente útil en búsquedas frecuentes, como cuando se buscan libros por autor o ISBN, o se listan préstamos por usuario o estado.

Además, los índices son cruciales en columnas que son claves foráneas, ya que aceleran las operaciones de unión (JOIN) entre tablas, manteniendo la integridad referencial y optimizando la recuperación de datos relacionados.

Consultas SQL - Nivel 0 - Selección

Consultas acciones básicas en SQL:

```
-- Consulta de Selección SELECT
SELECT campo1, campo2, ..., campoN -- Lo que quiero mostrar en la consulta
FROM tabla1                        -- De donde lo voy a obtener
WHERE campo1 = 'valor de búsqueda' -- Condición que debe cumplir el registro
ORDER BY campo1;                  -- Opcional, si quiere un orden específico

/* ¿Puedo hacer una consulta sobre 2 o mas tablas? */
SELECT *                          -- Muestre todas las columnas
FROM LIBRO, AUTOR                 -- De las tablas Autor y Libro
WHERE LIBRO.id_autor = AUTOR.id_autor; -- Cuando el id de Autor corresponda con el del Libro
-- Cuando se efectua una selección de mas de una tabla por campos en común se le llama INNER JOIN

-- ¿Puedo contar los registros de una tabla?
SELECT count(*) -- La funcion count entregará un valor escalar
FROM AUTOR;    -- Desde la tabla de autores.
```

El símbolo ";" le indica al interprete de SQL el termino de la consulta.

Consultas SQL - Nivel 0 - Inserción

Consultas acciones básicas de manipulación de datos en SQL:

-- Consulta o Peteción de Inserción INSERT

INSERT INTO

-- Acción de Insertar

tabla1 (campo1, campo2, ..., campoN)

-- En donde debe insertar

VALUES ('valor campo1', 'valor campo2', ..., ' valor campoN'); -- Valores

-- Ejemplo inserción de un Autor

INSERT INTO

AUTOR (id_autor, nombre, apellido, fecha_nacimiento)

VALUES (1, 'Gabriel', 'García Márquez', TO_DATE('1927-03-06', 'YYYY-MM-DD'));

-- Usa funcion de manipulación para campo de tipo de dato de fechas (DATE)

-- IMPORTANTE: Cuando manipule registros una vez acabado no olvide dar la instrucción COMMIT

COMMIT; -- Esta instrucción le dice al motor de base de datos que haga efectivos los cambios instruidos

Consultas SQL - Nivel 0 - Actualización

Consultas acciones básicas actualización en SQL:

```
-- Consulta de actualización UPDATE
UPDATE tabla1          -- Acción de Actualizar y sobre que tabla
SET campo1 = 'nuevo valor campo1', -- Se indica cada campo que se debe actualizar
    campo2 = 'nuevo valor campo2',  -- Se asigna cada campo el nuevo valor que debe tomar
    ...,
    campoN = 'nuevo valorN'
WHERE campo1 = 'valor de busqueda'; -- Condición que debe cumplir el registro para ser actualizado
/* IMPORTANTE: La condición es opcional pero se debe ser muy cuidadoso con los filtros */

/* Revise esta consulta */
UPDATE PRESTAMO
SET estado = 'DEVUELTO';
-- En este caso usted mantiene su empleo si no efectuó COMMIT

ROLLBACK; -- Instrucción Rollback deshace los cambios antes del ultimo COMMIT realizado por su usuario
```

Consultas SQL - Nivel 0 - Eliminación

Consultas acciones básicas en SQL:

-- Consulta de actualización UPDATE

DELETE tabla1 -- Acción de Eliminar registros y sobre cual es la tabla impactada

WHERE campo1 = 'valor de busqueda'; -- Condición que debe cumplir el registro para ser eliminado

/* IMPORTANTE: La condición es opcional pero se debe ser muy cuidadoso con los filtros

/* Revise esta consulta */

DELETE PRESTAMO

WHERE id_prestamo = id_prestamo;

-- En este caso nuevamente usted mantiene su empleo si no efectuó COMMIT

ROLLBACK; -- Instrucción Rollback deshace los cambios antes del ultimo COMMIT realizado por su usuario

Consultas SQL - Parte 1: Búsquedas Básicas

Muestra ejemplos de consultas SQL básicas:

-- Consulta 1: Listar todos los libros con información del autor

```
SELECT l.id_libro, l.titulo, a.nombre, a.apellido, l.año_publicacion  
FROM LIBRO l  
JOIN AUTOR a ON l.id_autor = a.id_autor  
ORDER BY l.titulo;
```

-- Consulta 2: Buscar libros disponibles

```
SELECT id_libro, titulo, cantidad_disponible  
FROM LIBRO  
WHERE cantidad_disponible > 0  
ORDER BY titulo;
```

-- Consulta 3: Listar usuarios registrados en el último mes

```
SELECT id_usuario, nombre, email, fecha_registro  
FROM USUARIO  
WHERE fecha_registro >= ADD_MONTHS(SYSDATE, -1)  
ORDER BY fecha_registro DESC;
```

Explica cada consulta y su propósito en el sistema de gestión de biblioteca.

Consultas SQL - Parte 2: Análisis y Reportes

Muestra consultas más complejas para análisis:

-- Consulta 4: Contar préstamos activos por usuario

```
SELECT u.id_usuario, u.nombre, COUNT(p.id_prestamo) AS prestamos_activos
FROM USUARIO u
LEFT JOIN PRESTAMO p ON u.id_usuario = p.id_usuario AND p.estado = 'ACTIVO'
GROUP BY u.id_usuario, u.nombre
HAVING COUNT(p.id_prestamo) > 0
ORDER BY prestamos_activos DESC;
```

-- Consulta 5: Libros más prestados

```
SELECT l.id_libro, l.titulo, COUNT(p.id_prestamo) AS total_prestamos
FROM LIBRO l
LEFT JOIN PRESTAMO p ON l.id_libro = p.id_libro
GROUP BY l.id_libro, l.titulo
ORDER BY total_prestamos DESC;
```

Consultas SQL - Parte 2: Análisis y Reportes

```
-- Consulta 6: Préstamos vencidos (sin devolución)
SELECT p.id_prestamo, u.nombre, l.titulo, p.fecha_prestamo
FROM PRESTAMO p
JOIN USUARIO u ON p.id_usuario = u.id_usuario
JOIN LIBROS l ON p.id_libro = l.id_libro
WHERE p.estado = 'ACTIVO' AND p.fecha_prestamo < SYSDATE - 30;
```

Estas consultas utilizan combinaciones avanzadas de agregaciones, cláusulas JOIN y condiciones para generar reportes útiles que brindan una visión profunda de las operaciones de la biblioteca. La Consulta 4, por ejemplo, utiliza LEFT JOIN y GROUP BY con HAVING para identificar usuarios con préstamos activos, lo que es crucial para la gestión de usuarios y la comunicación.

La Consulta 5 emplea LEFT JOIN y COUNT junto con GROUP BY para determinar los libros más populares, ayudando en la toma de decisiones sobre adquisiciones y promociones. Finalmente, la Consulta 6 combina múltiples JOINs y una condición de fecha para localizar préstamos vencidos, permitiendo una acción proactiva para la recuperación de libros.

Estas consultas son fundamentales para el análisis de datos, la identificación de tendencias y la optimización de los servicios de la biblioteca.

Consultas SQL - Parte 3: Análisis y Reportes

Considere que estas dos consultas en el fondo obtienen el mismo resultado solo se escriben de forma distinta.

```
SELECT p.id_prestamo, u.nombre, l.titulo,  
p.fecha_prestamo  
FROM PRESTAMO p  
JOIN USUARIO u ON p.id_usuario = u.id_usuario  
JOIN LIBROS l ON p.id_libro = l.id_libro  
WHERE p.estado = 'ACTIVO' AND p.fecha_prestamo <  
SYSDATE - 30;
```

```
SELECT p.id_prestamo, u.nombre, l.titulo,  
p.fecha_prestamo  
FROM PRESTAMO p, USUARIO u, LIBROS l  
WHERE p.id_usuario = u.id_usuario  
AND p.id_libro = l.id_libro  
AND p.estado = 'ACTIVO'  
AND p.fecha_prestamo < SYSDATE - 30;
```

Usted puede escribir las consultas como mejor la pueda interpretar.

Consultas SQL - Parte 3: Vistas

En muchas ocasiones tendrá que escribir consultas con frecuencia y eso es muy bueno, porque aprende bien el lenguaje y modelo de su base de datos, pero en algún momento puede requerir obtener respuestas rápidas. Intente ejecutar la siguiente instrucción para almacenar una consulta en la base de datos con un nombre:

```
CREATE VIEW VISTA_PRESTAMOS AS
SELECT p.id_prestamo, u.nombre, l.titulo, p.fecha_prestamo
FROM PRESTAMO p, USUARIO u, LIBROS l
WHERE p.id_usuario = u.id_usuario
AND p.id_libro = l.id_libro;
```

Para utilizar esta vista, utilice una consulta como si fuera una tabla corriente

```
SELECT vp.*
FROM VISTA_PRESTAMOS vp
WHERE vp.estado = 'ACTIVO'
AND vp.fecha_prestamo < SYSDATE - 30 --
```

Puede agregar también filtros sobre los campos que trae la vista.

Procedimientos Almacenados - Parte 1: Inserción de Datos

Muestra procedimientos almacenados para operaciones básicas:

```
-- Procedimiento para registrar un nuevo usuario
CREATE OR REPLACE PROCEDURE registrar_usuario(
  p_id_usuario IN NUMBER,
  p_nombre IN VARCHAR2,
  p_email IN VARCHAR2,
  p_telefono IN VARCHAR2
) AS
BEGIN
  INSERT INTO USUARIO (id_usuario, nombre, email, telefono, fecha_registro)
  VALUES (p_id_usuario, p_nombre, p_email, p_telefono, SYSDATE);
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Usuario registrado exitosamente');
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Error: El usuario ya existe');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
  ROLLBACK;
END registrar_usuario;
/

-- Procedimiento para registrar un nuevo libro
CREATE OR REPLACE PROCEDURE registrar_libro(
  p_id_libro IN NUMBER,
  p_titulo IN VARCHAR2,
  p_id_autor IN NUMBER,
  p_isbn IN VARCHAR2,
  p_año IN NUMBER,
  p_cantidad IN NUMBER
) AS
BEGIN
  INSERT INTO LIBRO (id_libro, titulo, id_autor, isbn, año_publicacion, cantidad_disponible)
  VALUES (p_id_libro, p_titulo, p_id_autor, p_isbn, p_año, p_cantidad);
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Libro registrado exitosamente');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
  ROLLBACK;
END registrar_libro;
/
```

Estos procedimientos almacenados, `registrar_usuario` y `registrar_libro`, están diseñados para encapsular y estandarizar las operaciones de inserción de datos en las tablas `USUARIOS` y `LIBROS`, respectivamente. Ambos siguen una estructura similar, definida por `CREATE OR REPLACE PROCEDURE`, que permite crear o actualizar el procedimiento. Los parámetros, como `p_id_usuario` y `p_nombre`, son de entrada (IN) y reciben los valores necesarios para la inserción.

Dentro del bloque `BEGIN...END`, se realiza la operación `INSERT` para añadir una nueva fila a la tabla correspondiente. Después de una inserción exitosa, la transacción se confirma con `COMMIT`;, asegurando que los cambios sean permanentes en la base de datos. Ambos procedimientos incluyen un robusto manejo de excepciones mediante el bloque `EXCEPTION`. El procedimiento `registrar_usuario` maneja específicamente la excepción `DUP_VAL_ON_INDEX` para cuando se intenta insertar un usuario con un email ya existente (debido a la restricción `UNIQUE`). En caso de cualquier otro error (`WHEN OTHERS THEN`), se muestra el mensaje de error (`SQLERRM`) y se revierte la transacción con `ROLLBACK`;, garantizando la integridad de los datos.

Procedimientos Almacenados - Parte 2: Gestión de Préstamos

Muestra procedimientos para operaciones de préstamos:

```
-- Procedimiento para registrar un préstamo
CREATE OR REPLACE PROCEDURE registrar_prestamo(
  p_id_prestamo IN NUMBER,
  p_id_usuario IN NUMBER,
  p_id_libro IN NUMBER
) AS
  v_cantidad NUMBER;
BEGIN
  -- Verificar disponibilidad del libro
  SELECT cantidad_disponible INTO v_cantidad
  FROM LIBRO
  WHERE id_libro = p_id_libro;
  IF v_cantidad > 0 THEN
    INSERT INTO PRESTAMO (id_prestamo, id_usuario, id_libro, fecha_prestamo, estado)
    VALUES (p_id_prestamo, p_id_usuario, p_id_libro, SYSDATE, 'ACTIVO');
    UPDATE LIBRO SET cantidad_disponible = cantidad_disponible - 1
    WHERE id_libro = p_id_libro;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Préstamo registrado exitosamente');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Error: Libro no disponible');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Libro no encontrado');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    ROLLBACK;
END registrar_prestamo;
/

-- Procedimiento para registrar devolución
CREATE OR REPLACE PROCEDURE registrar_devolucion(
  p_id_prestamo IN NUMBER
) AS
  v_id_libro NUMBER;
BEGIN
  UPDATE PRESTAMO SET estado = 'DEVUELTO', fecha_devolucion = SYSDATE
  WHERE id_prestamo = p_id_prestamo
  RETURNING id_libro INTO v_id_libro;
  UPDATE LIBRO SET cantidad_disponible = cantidad_disponible + 1
  WHERE id_libro = v_id_libro;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Devolución registrada exitosamente');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    ROLLBACK;
END registrar_devolucion;
/
```

Explica cómo estos procedimientos manejan la lógica de negocio, validaciones y transacciones complejas.

Procedimientos Almacenados - Parte 3: Funciones y Reportes

Muestra funciones y procedimientos para reportes:

```
-- Función para obtener el total de préstamos activos de un usuario
CREATE OR REPLACE FUNCTION contar_prestamos_activos(
  p_id_usuario IN NUMBER
) RETURN NUMBER AS
  v_total NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_total
  FROM PRESTAMO
  WHERE id_usuario = p_id_usuario AND estado = 'ACTIVO';
  RETURN v_total;
EXCEPTION
  WHEN OTHERS THEN
    RETURN 0;
END contar_prestamos_activos;
/

-- Procedimiento para generar reporte de préstamos vencidos
CREATE OR REPLACE PROCEDURE reporte_prestamos_vencidos AS
  CURSOR c_prestamos_vencidos IS
    SELECT p.id_prestamo, u.nombre, l.titulo, p.fecha_prestamo,
      TRUNC(SYSDATE - p.fecha_prestamo) AS dias_vencido
    FROM PRESTAMO p
    JOIN USUARIO u ON p.id_usuario = u.id_usuario
    JOIN LIBRO l ON p.id_libro = l.id_libro
    WHERE p.estado = 'ACTIVO' AND p.fecha_prestamo < SYSDATE - 30
    ORDER BY dias_vencido DESC;
BEGIN
  DBMS_OUTPUT.PUT_LINE('=== REPORTE DE PRÉSTAMOS VENCIDOS ===');
  DBMS_OUTPUT.PUT_LINE('Fecha: ' || TO_CHAR(SYSDATE, 'DD/MM/YYYY'));
  DBMS_OUTPUT.PUT_LINE('');
  FOR v_prestamo IN c_prestamos_vencidos LOOP
    DBMS_OUTPUT.PUT_LINE('ID Préstamo: ' || v_prestamo.id_prestamo);
    DBMS_OUTPUT.PUT_LINE('Usuario: ' || v_prestamo.nombre);
    DBMS_OUTPUT.PUT_LINE('Libro: ' || v_prestamo.titulo);
    DBMS_OUTPUT.PUT_LINE('Días vencido: ' || v_prestamo.dias_vencido);
    DBMS_OUTPUT.PUT_LINE('---');
  END LOOP;
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END reporte_prestamos_vencidos;
/
```

Explica el uso de funciones, cursores y procedimientos para generar reportes complejos.

Stored Procedures - Parte 4: Ejecución de Procedimientos y Funciones

Como podemos ejecutar procedimientos y funciones:

-- Ejecución de Función para obtener el total de préstamos activos de un usuario

```
SELECT contar_prestamos_activos(5)  
FROM DUAL;
```

-- Procedimiento para generar reporte de préstamos vencidos

```
BEGIN  
    reporte_prestamos_vencidos;  
END;
```

Stored Procedures - Parte 4: Ejecución de Procedimientos y Funciones

Ejecutar procedimientos con parámetros:

```
-- Ejecución de Función para obtener el total de préstamos activos de un usuario
DECLARE
  v_id_usuario NUMBER := 6;
  v_nombre   VARCHAR2(100) := 'Laura Fernández';
  v_email    VARCHAR2(100) := 'laura.fernandez@example.com';
  v_telefono VARCHAR2(15) := '555-1234';
BEGIN
  registrar_usuario(
    v_id_usuario,
    v_nombre,
    v_email,
    v_telefono
  );
END;
```

Objetos de BBDD - Parte 1: ¿Donde se almacena todo esto?

Oracle almacena todos los objetos creados por los usuarios en las tablas propias de Oracle. Examine el resultado de las siguientes consultas:

```
select * from all_tables;
```

```
select * from all_objects;
```

```
select * from all_source;
```

```
select * from user_source where object_name = 'user_source';
```

Estas son algunas tablas o internas de Oracle.

Oracle SQL Developer - Herramienta de Desarrollo

Oracle SQL Developer es un entorno de desarrollo integrado (IDE) gratuito y gráfico que simplifica el desarrollo de bases de datos y la administración de datos en Oracle Database, tanto en entornos tradicionales como en la nube. Ofrece una potente interfaz para desarrolladores de SQL y PL/SQL, facilitando tareas cruciales del día a día.



Edición y Ejecución de Scripts

Facilita la escritura, edición y ejecución de sentencias SQL, scripts y bloques PL/SQL. Incluye funcionalidades de autocompletado y formateo.



Gestión de Conexiones

Permite crear y administrar conexiones a múltiples bases de datos Oracle, así como a bases de datos de terceros, con soporte para diversas configuraciones.



Depuración de Procedimientos

Ofrece herramientas avanzadas para depurar procedimientos, funciones y paquetes PL/SQL, permitiendo el control del flujo, puntos de interrupción e inspección de variables.



Explorador de Objetos

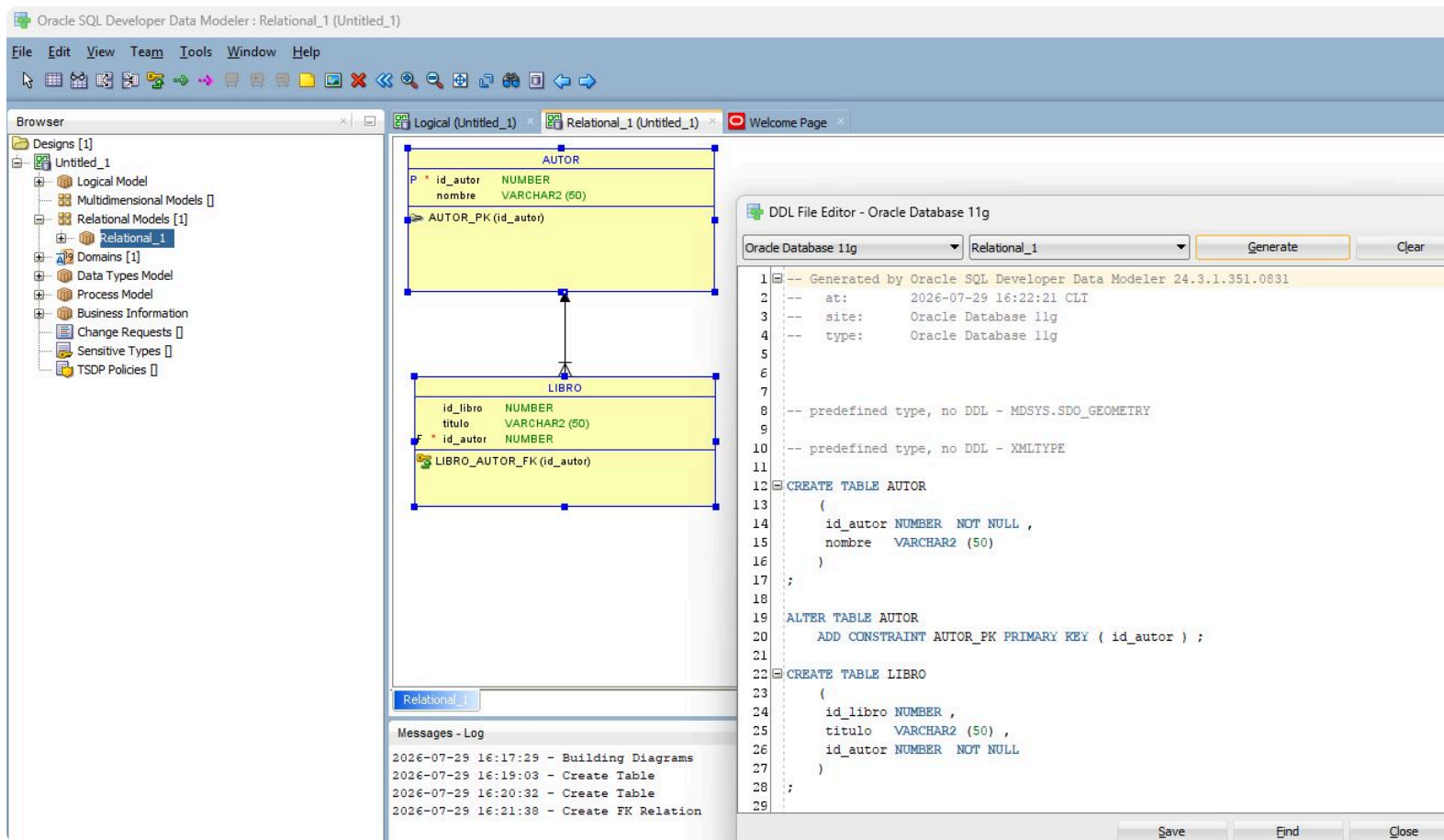
Proporciona una interfaz intuitiva para navegar, visualizar y gestionar esquemas, tablas, vistas, índices y otros objetos de la base de datos.

Esta herramienta es completamente gratuita y puede descargarse directamente desde el sitio web oficial de Oracle, lo que la convierte en una opción accesible y robusta para cualquier desarrollador de bases de datos.

[Oracle SQL Developer Downloads](#)

Oracle SQL Developer - Data Modeler

Oracle SQL Developer Data Modeler es una herramienta de modelamiento gratuito y gráfico que permite construir modelos relacionales que permite generar las instrucciones SQL para crear los objetos de una base de datos Oracle. Este modelo considera tablas, relaciones indices, claves primarias y foraneas.



Esta herramienta es completamente gratuita y puede descargarse directamente desde el sitio web oficial de Oracle, lo que la convierte en una opción accesible y robusta para cualquier desarrollador de bases de datos.

[SQL Developer Data Modeler Downloads](#)



Ejercicios Prácticos - Paso a Paso

Para consolidar los conocimientos adquiridos, se propone la siguiente serie de ejercicios prácticos que guiarán a los participantes a través de la implementación de una base de datos de biblioteca desde cero.

1

Crear Modelo de Datos

Diseñar y crear las tablas de la base de datos (LIBROS, USUARIOS, PRESTAMOS).

2

Insertar Datos de Prueba

Poblar las tablas con datos ficticios para simular un entorno real.

3

Crear Índices

Optimizar el rendimiento de las consultas mediante la creación de índices adecuados.

4

Ejecutar Consultas Básicas

Realizar consultas SELECT, INSERT, UPDATE, DELETE para manipular los datos.

5

Crear Procedimientos Almacenados

Implementar la lógica de negocio en PL/SQL para registrar préstamos y devoluciones.

6

Ejecutar y Validar Procedimientos

Invocar los procedimientos almacenados y verificar su correcto funcionamiento.

7

Generar Reportes

Desarrollar funciones y procedimientos para generar reportes como el de préstamos vencidos.

Cada uno de estos ejercicios debe ser completado utilizando la herramienta **Oracle SQL Developer** para familiarizarse con su interfaz y funcionalidades.

Mejores Prácticas en PL/SQL

Adoptar estas directrices asegura la creación de código PL/SQL robusto, mantenible y eficiente.

1

Nombres Descriptivos

Utiliza nombres claros para variables y procedimientos, mejorando la legibilidad.

2

Manejo de Excepciones

Implementa bloques EXCEPTION para gestionar errores y evitar caídas inesperadas.

3

Documentación Clara

Añade comentarios para explicar la lógica compleja y las decisiones de diseño.

4

Transacciones Adecuadas

Usa COMMIT y ROLLBACK para mantener la integridad de los datos en todo momento.

5

Validación de Datos

Verifica siempre los datos de entrada para prevenir errores y vulnerabilidades.

6

Optimización de Consultas

Emplea índices y técnicas de optimización para mejorar el rendimiento.

7

Lógica en PL/SQL

Centraliza la lógica de negocio compleja en procedimientos almacenados.

8

Pruebas Exhaustivas

Realiza pruebas rigurosas para asegurar la funcionalidad y estabilidad del código.

Conclusión y Recursos

Este taller ha sido una inmersión profunda en el mundo de PL/SQL. Aquí te dejamos un resumen y los próximos pasos para continuar tu aprendizaje.

- 1 Resumen del Taller**

Exploramos desde la sintaxis básica de PL/SQL hasta la creación de funciones, procedimientos, paquetes y el manejo robusto de excepciones, aplicado a una base de datos de biblioteca.
- 2 Importancia de PL/SQL**

PL/SQL es fundamental para desarrollar lógica de negocio compleja y optimizar el rendimiento en Oracle Database, garantizando aplicaciones eficientes y mantenibles.
- 3 Recursos Adicionales**

Consulta la documentación oficial de Oracle, explora tutoriales avanzados en línea y participa en comunidades de desarrolladores para resolver dudas y aprender de otros.
- 4 Practica y Experimenta**

La mejor manera de dominar PL/SQL es mediante la práctica constante. Crea tus propios proyectos y escenarios para aplicar lo aprendido.
- 5 Contacto y Soporte**

Para cualquier pregunta adicional o asistencia, no dudes en contactar a nuestro equipo de soporte. Estamos aquí para ayudarte en tu camino.

Preguntas Frecuentes (FAQ)

Aquí respondemos a algunas de las dudas más comunes sobre PL/SQL para ayudarte en tu aprendizaje.

1

Función vs. Procedimiento

Las **funciones** retornan un valor y pueden usarse en sentencias SQL. Los **procedimientos** ejecutan acciones y no siempre retornan un valor.

2

¿Cuándo usar Índices?

Usa índices en columnas frecuentemente consultadas (WHERE, JOIN, ORDER BY) para acelerar las búsquedas y mejorar el rendimiento de lectura de datos.

3

Manejo de Errores

Implementa bloques `EXCEPTION` para capturar y gestionar errores específicos o generales (`WHEN OTHERS`), evitando interrupciones inesperadas.

4

¿Qué es y cuándo usar un Cursor?

Un cursor es un puntero a un conjunto de filas resultante de una consulta. Se usa para procesar estas filas individualmente.

5

Optimización de Consultas

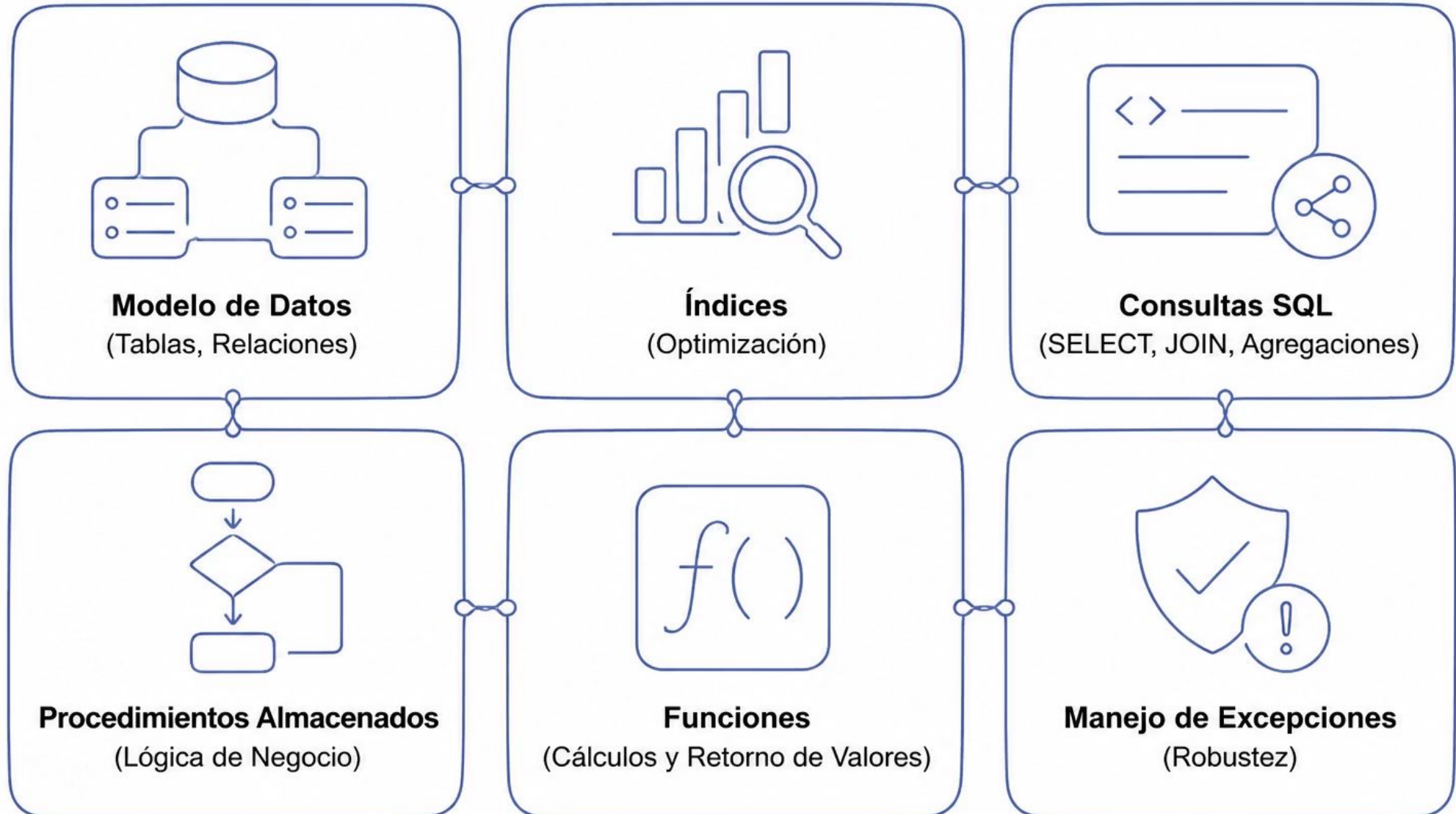
Usa índices, evita `SELECT *`, escribe cláusulas `WHERE` eficientes y realiza uniones (JOINS) adecuadas para mejorar el rendimiento.

6

PL/SQL en Aplicaciones Web

Sí, PL/SQL se puede usar en aplicaciones web, especialmente a través de Oracle Application Express (APEX) o mediante frameworks que interactúan con la base de datos.

Resumen del Taller - Componentes Clave



¡Gracias por tu participación!

Esperamos que este taller haya sido de gran valor para tu desarrollo profesional en PL/SQL. La clave para dominar cualquier tecnología es la práctica constante y la aplicación de los conocimientos adquiridos.

Sigue Practicando

No dejes de experimentar con Oracle SQL Developer y crea tus propios proyectos para reforzar lo aprendido.

Explora Recursos Avanzados

Consulta la documentación oficial de Oracle y explora cursos avanzados para profundizar tus conocimientos.

Conéctate con la Comunidad

Únete a foros y grupos de desarrolladores de PL/SQL para compartir experiencias y resolver dudas.

Cualquier ayuda que necesites con PL/SQL puedes contactarme al correo c.gutierrezhartard@tcs.com